

Hybrid Optimization Using Evolutionary Computation and Reinforcement Learning

الحلول المثلى الهجينة باستخدام الحساب
الارتقائي والتعلم المعزز

Presented by: Hesham Magdy

Agenda

- Introduction
- Background & Related Work
- The Proposed Framework
- Experiments & Results Analysis
- Conclusion & Future Work

Introduction

- Evolutionary Computation

- optimizes a population of candidate solutions
- maximizes/minimizes a fitness function

- Reinforcement Learning

- optimizes a policy function that maps states to actions
- maximizes cumulative reward across an entire episode

- NeuroEvolution Deep Reinforcement Learning

- deep neural network acts as a policy function for the reinforcement learning
- evolutionary algorithms optimizes the weights of this neural network

Introduction

○ Challenges

- Requires expertise across several technical areas
 - Reinforcement learning
 - Evolutionary algorithms
 - Deep learning
 - Parallel computing
- Scalability
- Limited frameworks support

Introduction

- Introducing a new proposed framework with the following features:
 - Diversity
 - 34 different evolutionary algorithms
 - customizable neural network with 22 activation functions
 - 12 different reinforcement learning environments
 - Flexibility
 - Scalability
 - Ease-of-use with no-code setup

Introduction

- Conducting 4 different sets of experiments:
 - 6 evolutionary algorithms with 6 different environments
 - demonstrating the framework diversity
 - achieving competitive results with the best reported results in the literature
 - 4 population sizes with 6 different environments
 - demonstrating the framework scalability
 - 4 evolutionary algorithms with 6 different activation functions
 - demonstrating the framework diversity
 - 4 population sizes with different number of iteration on CPU vs GPU
 - demonstrating the framework scalability

Introduction

○ Publication

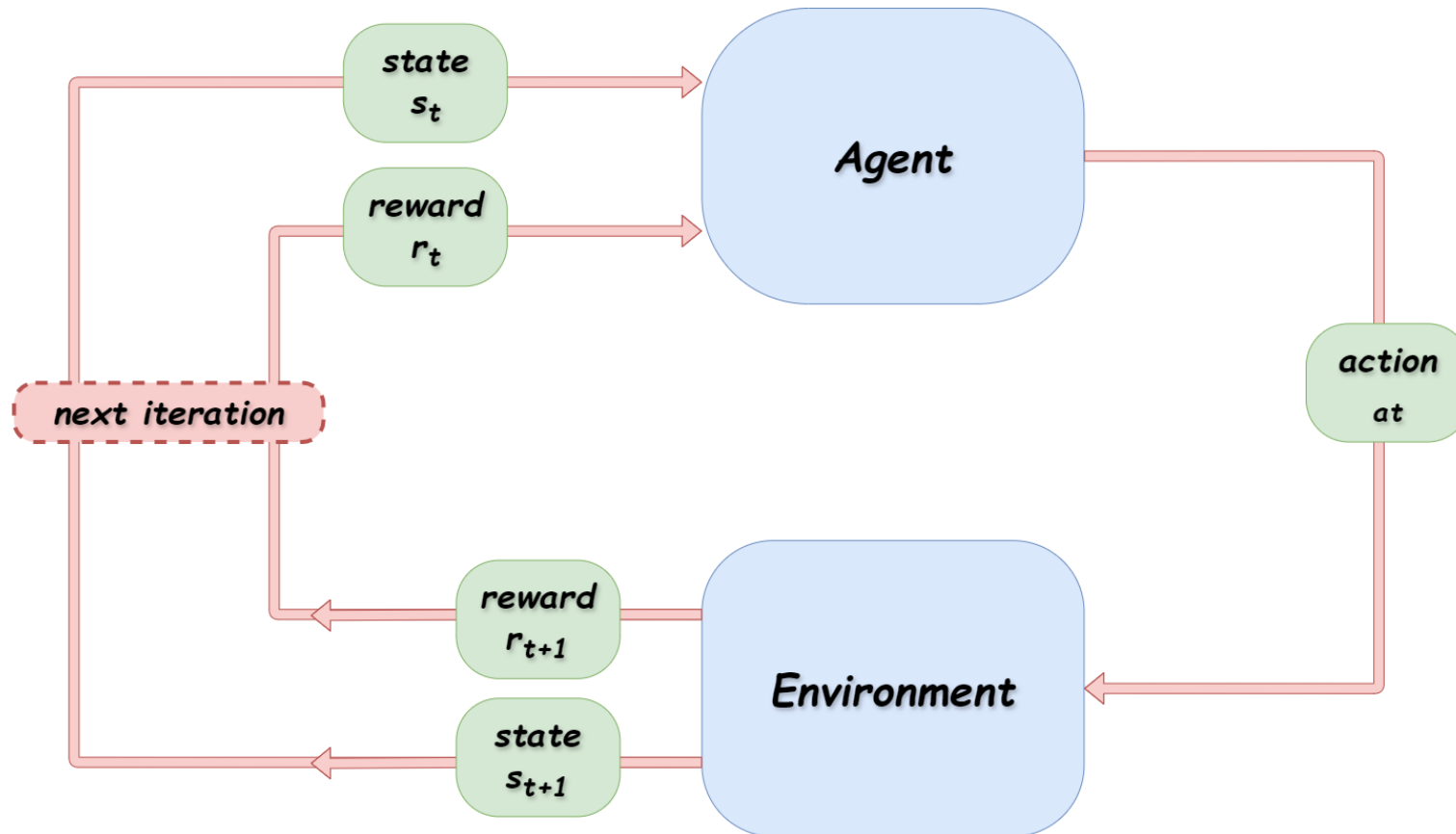
- “*Optimizing deep reinforcement learning through vectorized and parallel NeuroEvolution*”
- <https://doi.org/10.1016/j.future.2025.108334>
- “*Future Generation Computer Systems*”
 - Q1, Impact Factor: 6.1, CiteScore: 17.1

• Public Repository

- <https://github.com/Hesham-Magdy/EvoRL>

Background & Related Work

Reinforcement Learning

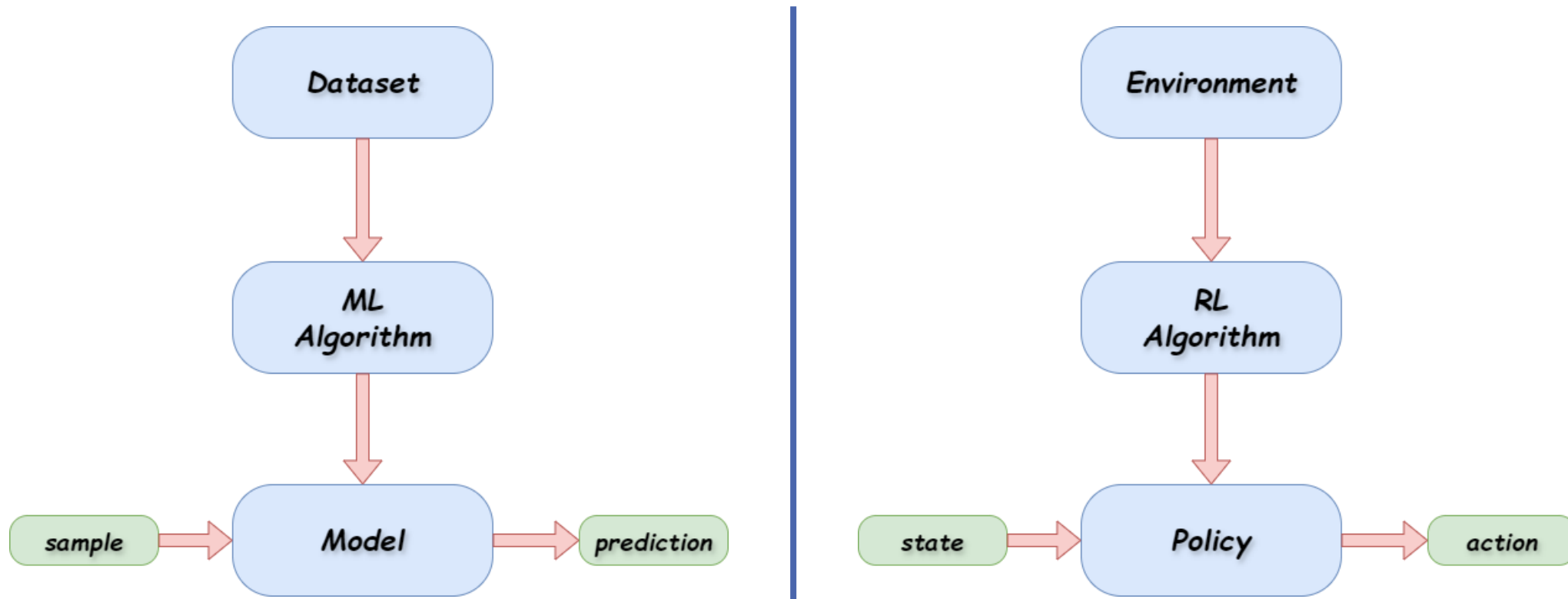


Reinforcement Learning

- Episode starts from the initial state and lasts until environment is terminated:
 - make a fatal mistake (losing a game)
 - positive living reward to encourage survivability
 - achieving a specific goal (winning a game)
 - negative living reward to encourage reaching the goal quickly
 - exceeding a limited number of actions
- Policy is the function the agent uses to map states to actions
- Agent aims to learn the policy that
 - maximizes the cumulative reward across entire episode

Reinforcement Learning

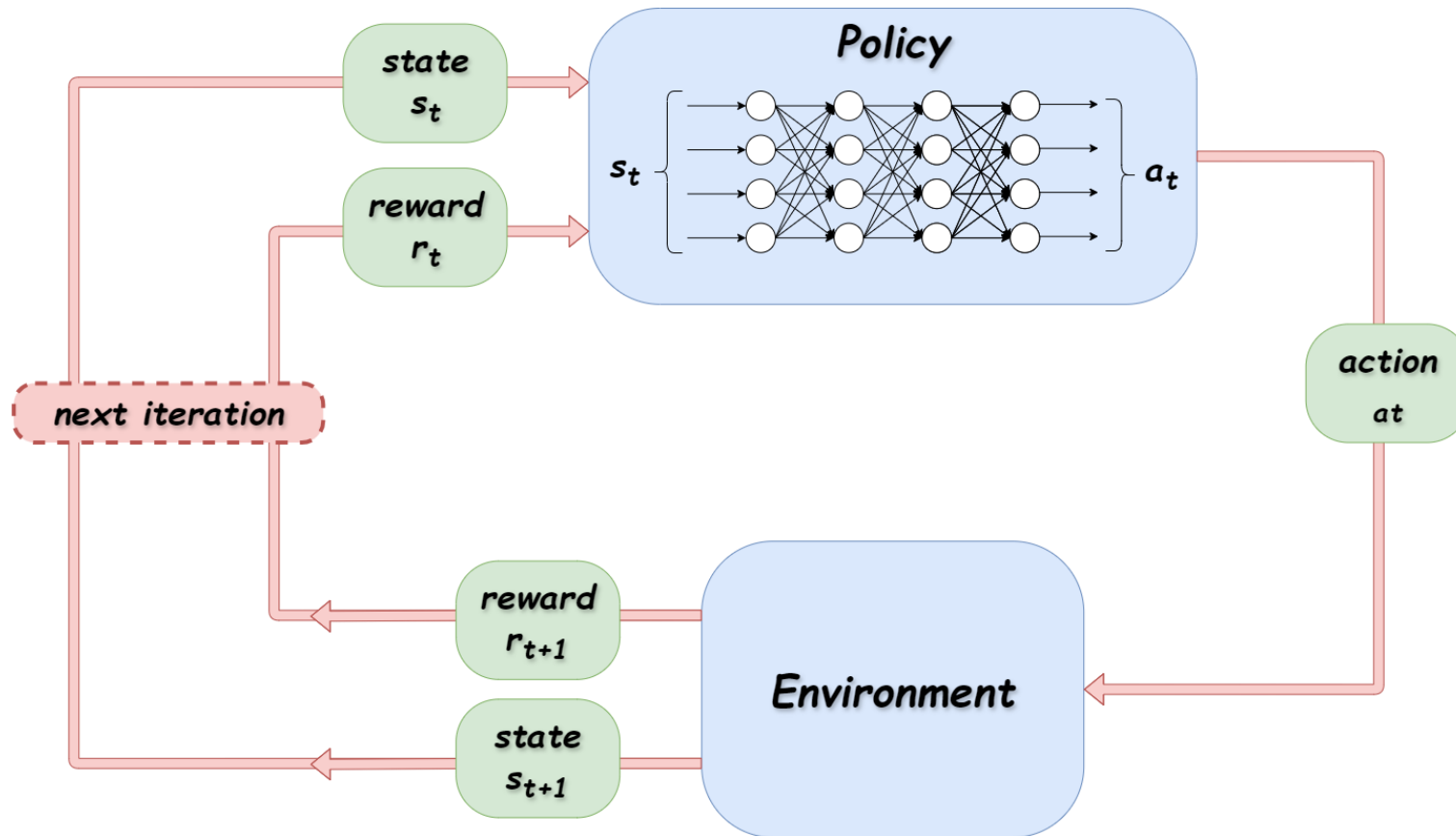
Machine Learning & Reinforcement Learning paradigms



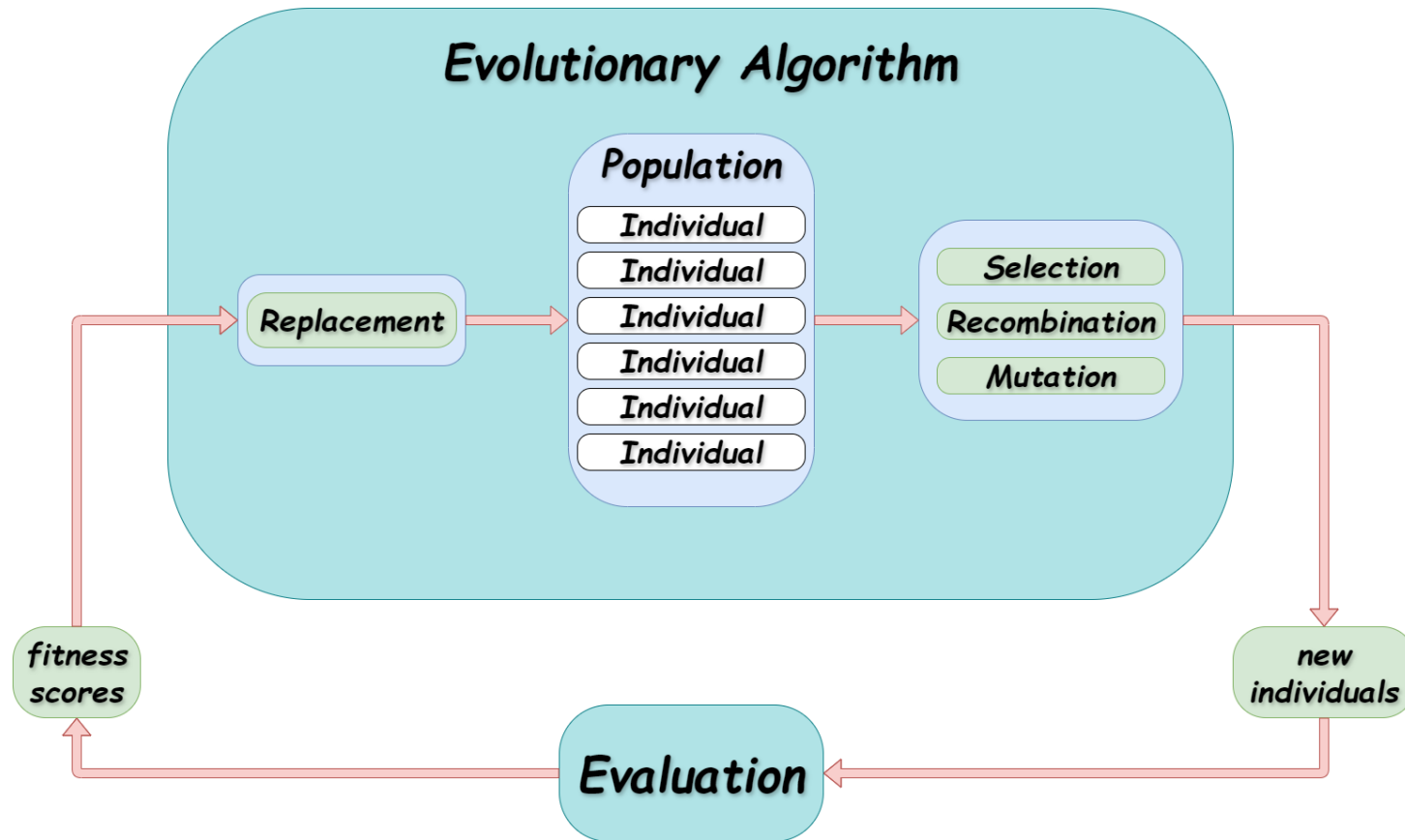
Reinforcement Learning

- Both control theory and reinforcement learning:
 - operate on a continuous feedback loop systems
 - determine the best action in response to a feedback signal
- In control theory:
 - system dynamics are well-defined
 - the system can be described as a mathematical model
- In reinforcement learning:
 - can handle scenarios where models are not available or difficult to derive
 - relies on direct interaction with the system rather than explicit modelling

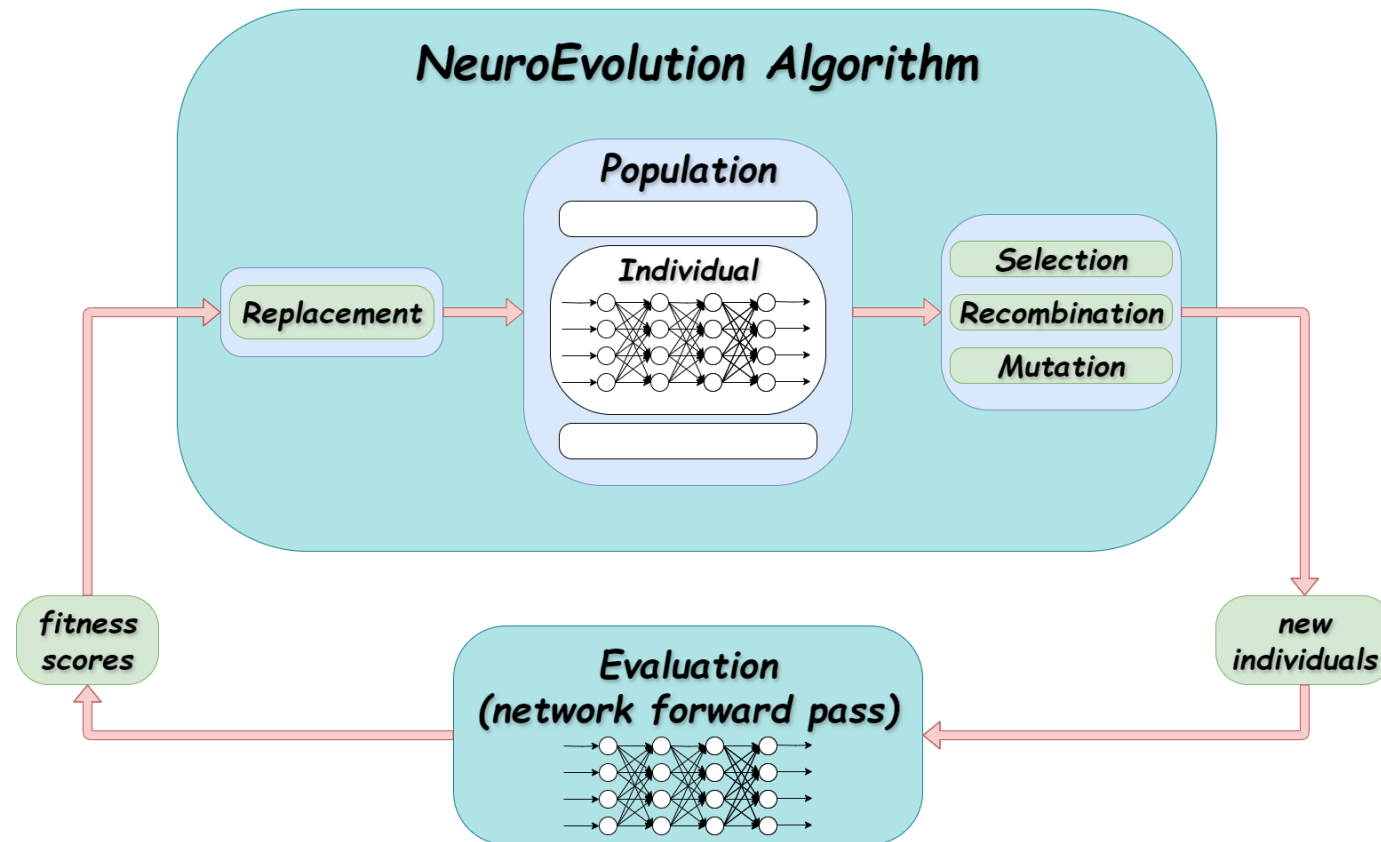
Deep Reinforcement Learning



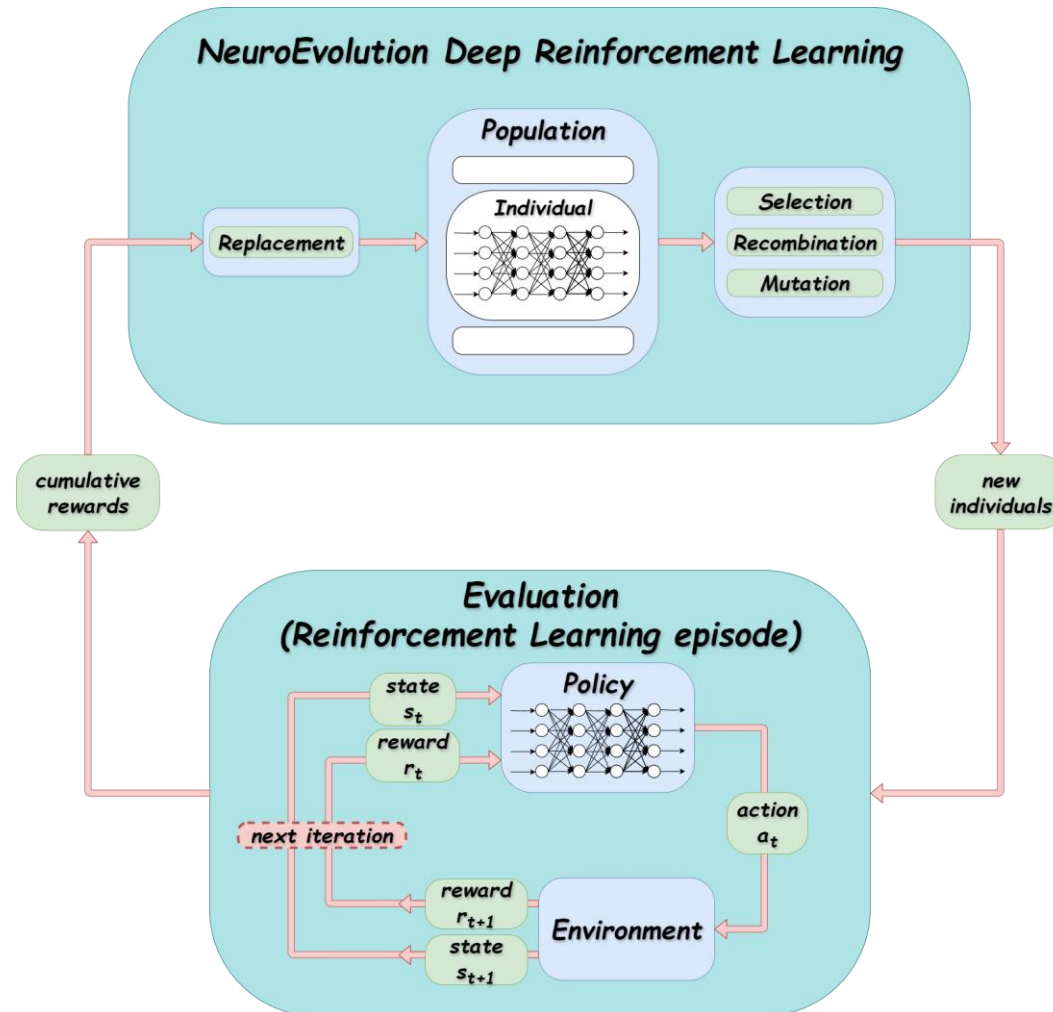
Evolutionary Algorithms



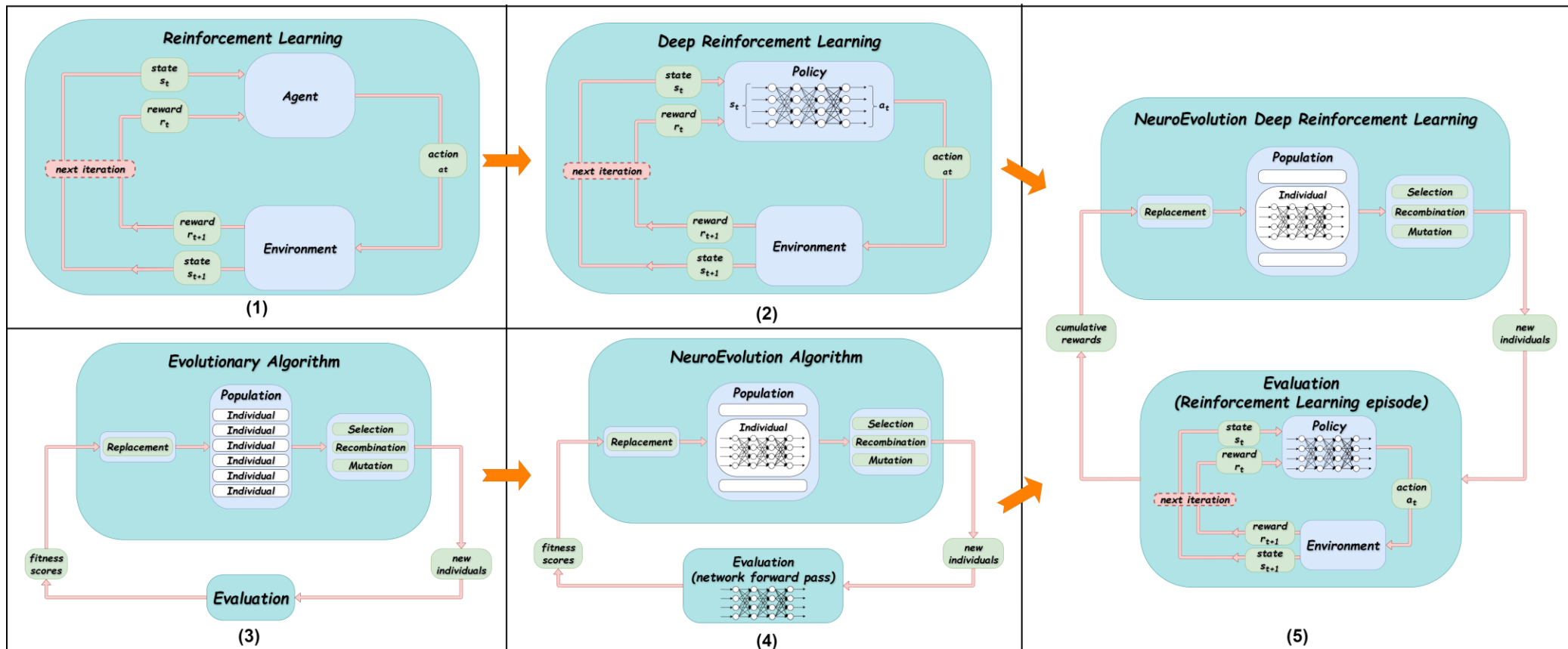
NeuroEvolution



NeuroEvolution Deep Reinforcement Learning



NeuroEvolution Deep Reinforcement Learning

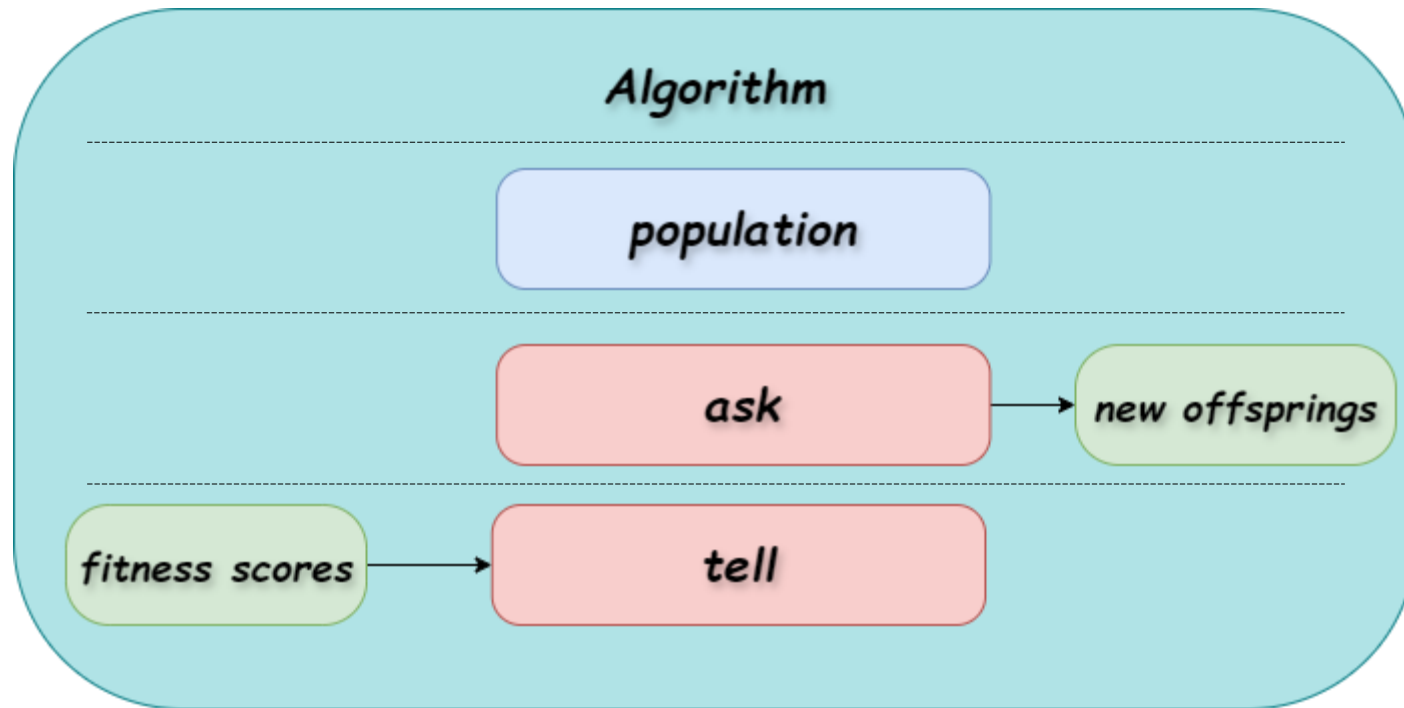


The Proposed Framework

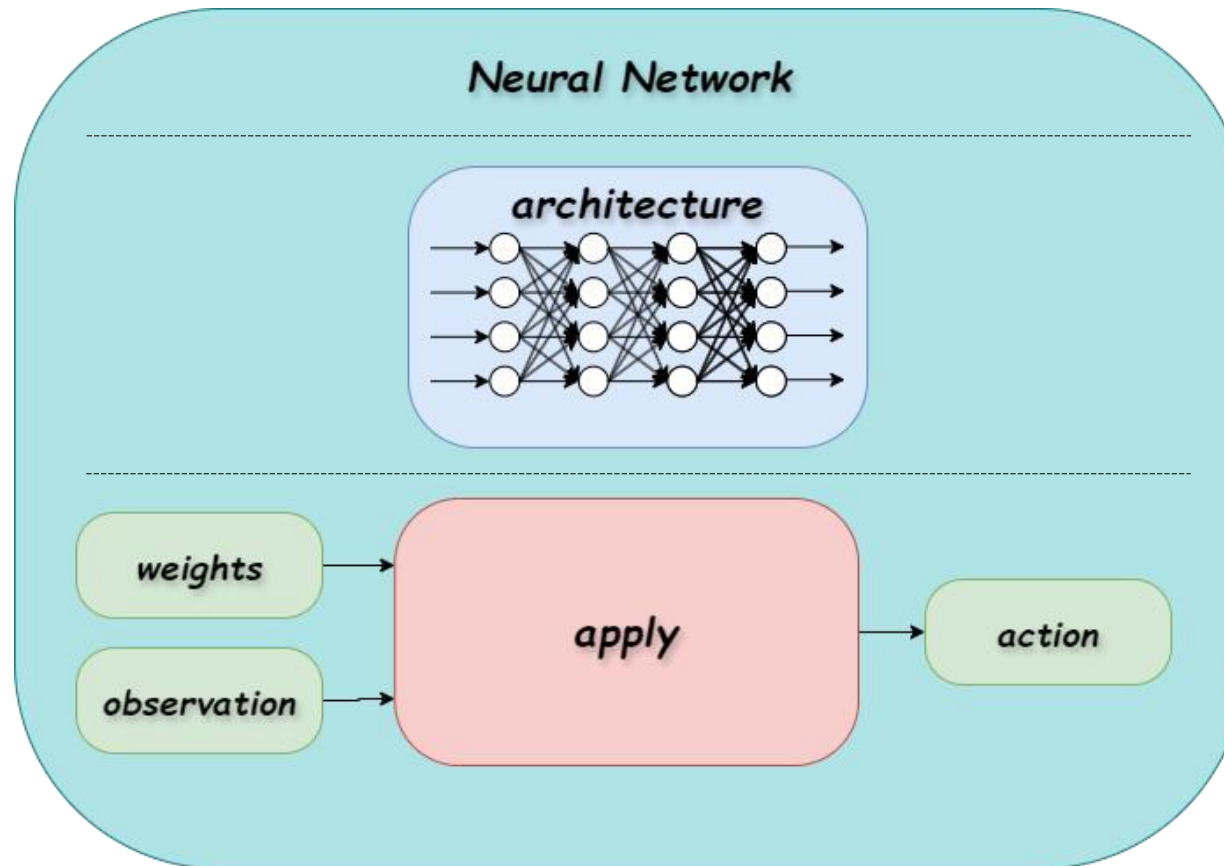
Framework Implementation

- Implementing a new proposed framework with the following features:
 - Diversity
 - 34 different algorithms, 12 different environments
 - customizable neural network with 22 activation functions
 - Flexibility
 - easy to add new components
 - Scalability
 - using JAX for vectorization
 - Ease-of-use with no-code setup
 - conducting experiments using simple configuration file instead of code

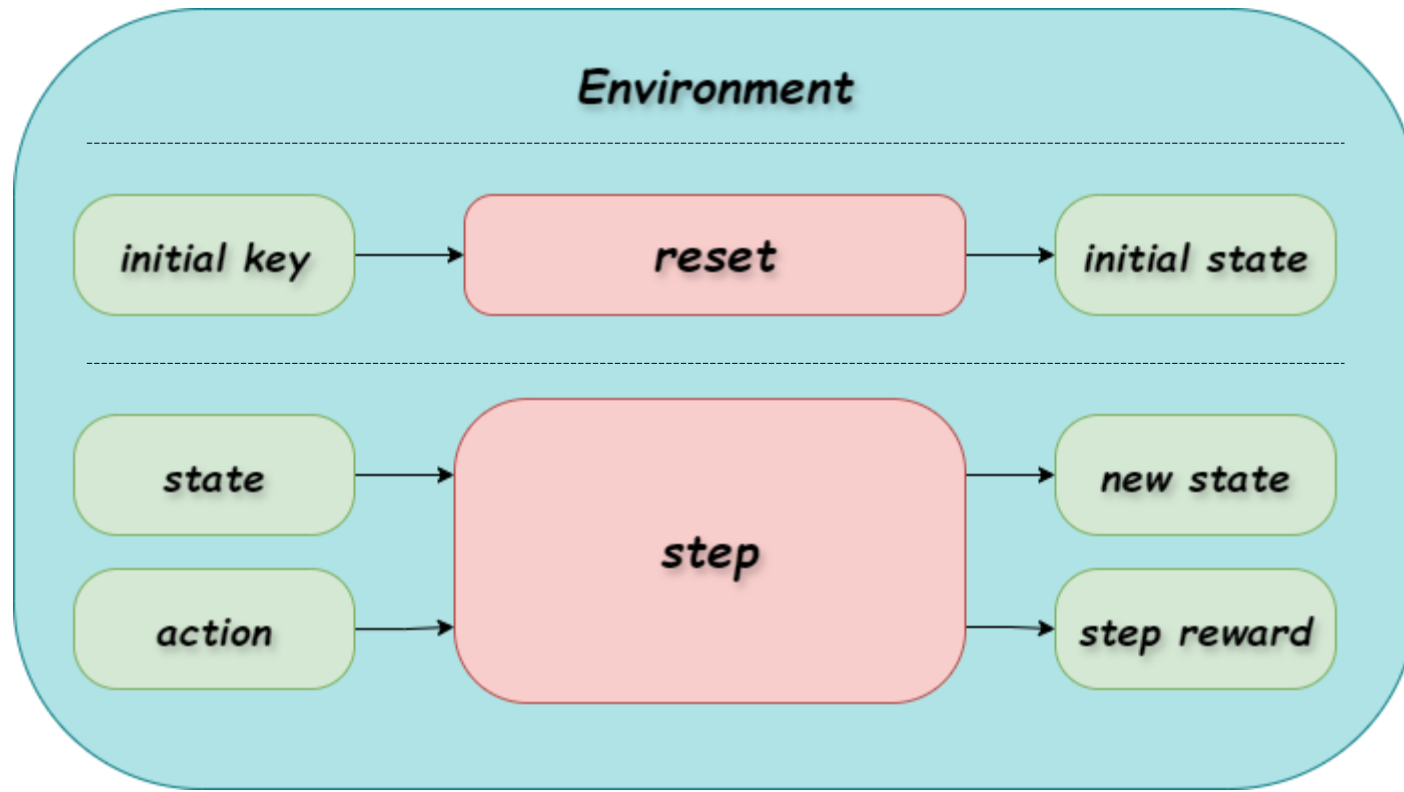
Framework Implementation



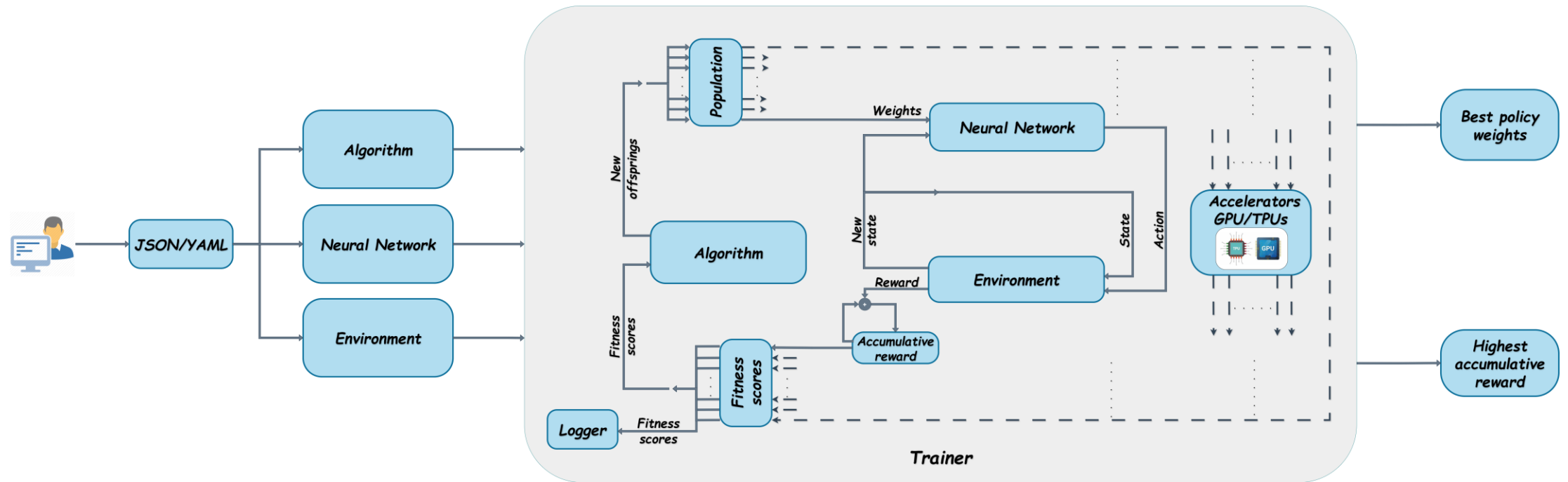
Framework Implementation



Framework Implementation



Framework Implementation



Framework Components

34 Implemented evolutionary algorithm in the proposed framework

• ARS	• GESMR-GA	• MA-ES	• RmES
• ASEBO	• GLD	• MR15-GA	• SAMR-GA
• BIPOP-CMA-ES	• GuidedES	• NoiseReuseES	• Sep-CMA-ES
• CMA-ES	• Indep- iAMaLGaM	• OpenES	• SimAnneal
• CR-FM-NES	• IPOP-CMA-ES	• PBT	• SimpleES
• DE	• LES	• PersistentES	• SimpleGA
• DES	• LGA	• PGPE	• SNES
• ESMC	• LM-MA-ES	• PSO	• xNES
• Full-iAMaLGaM			

Framework Components

22 Implemented activation function in the proposed framework

• CELU	• log-softmax	• softplus
• ELU	• log-sum-exp	• softsign
• GLU	• PReLU	• standardize
• GELU	• ReLU	• tanh
• hard-sigmoid	• ReLU6	
• hard-SiLU/ hard-Swish	• SELU	
• hard-tanh	• sigmoid	
• Leaky ReLU	• SiLU/ Swish	
• log-sigmoid	• softmax	

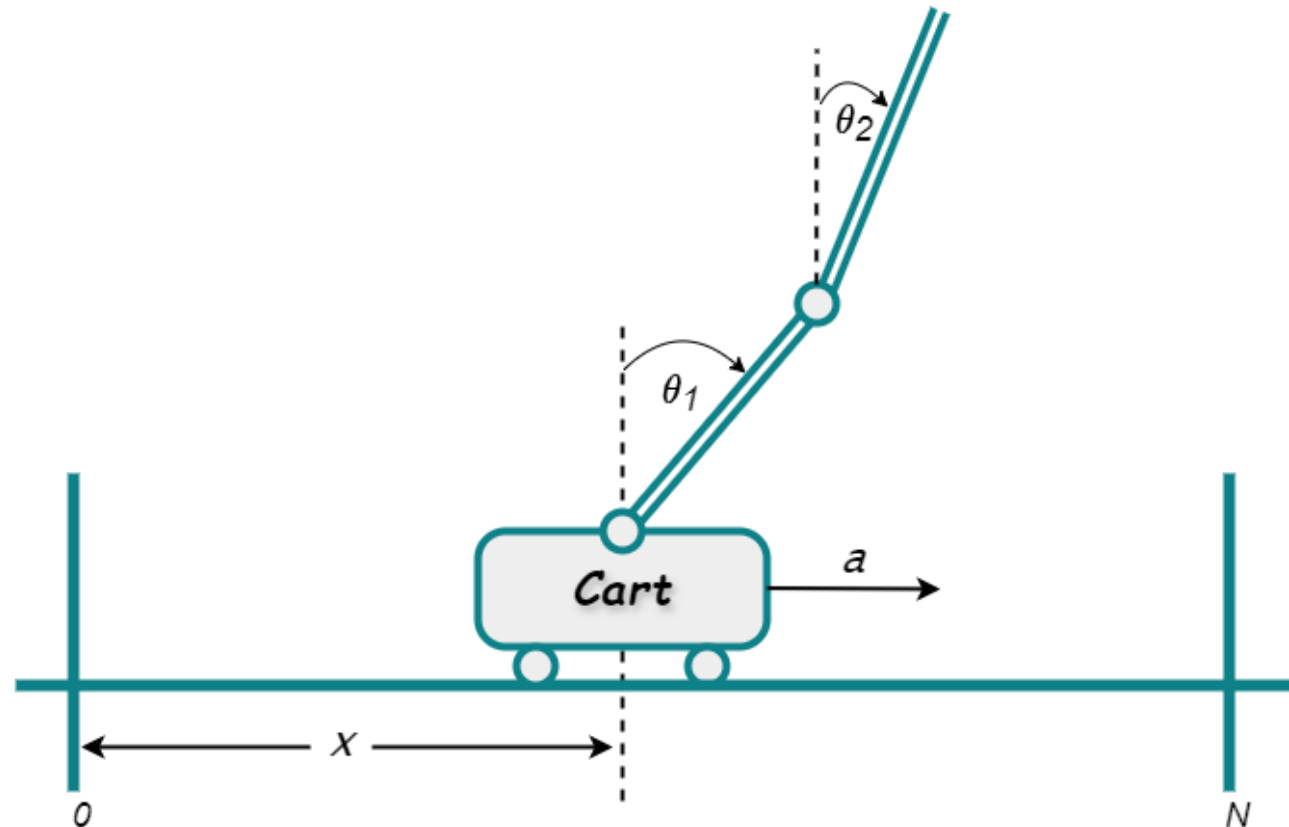
Framework Components

12 Implemented environment in the proposed framework

• CartPole	• Hopper
• Ant	• Pusher
• Humanoid	• Reacher
• HumanoidStandup	• Inverted Pendulum
• HalfCheetah	• Inverted Double Pendulum
• Walker2D	• Swimmer

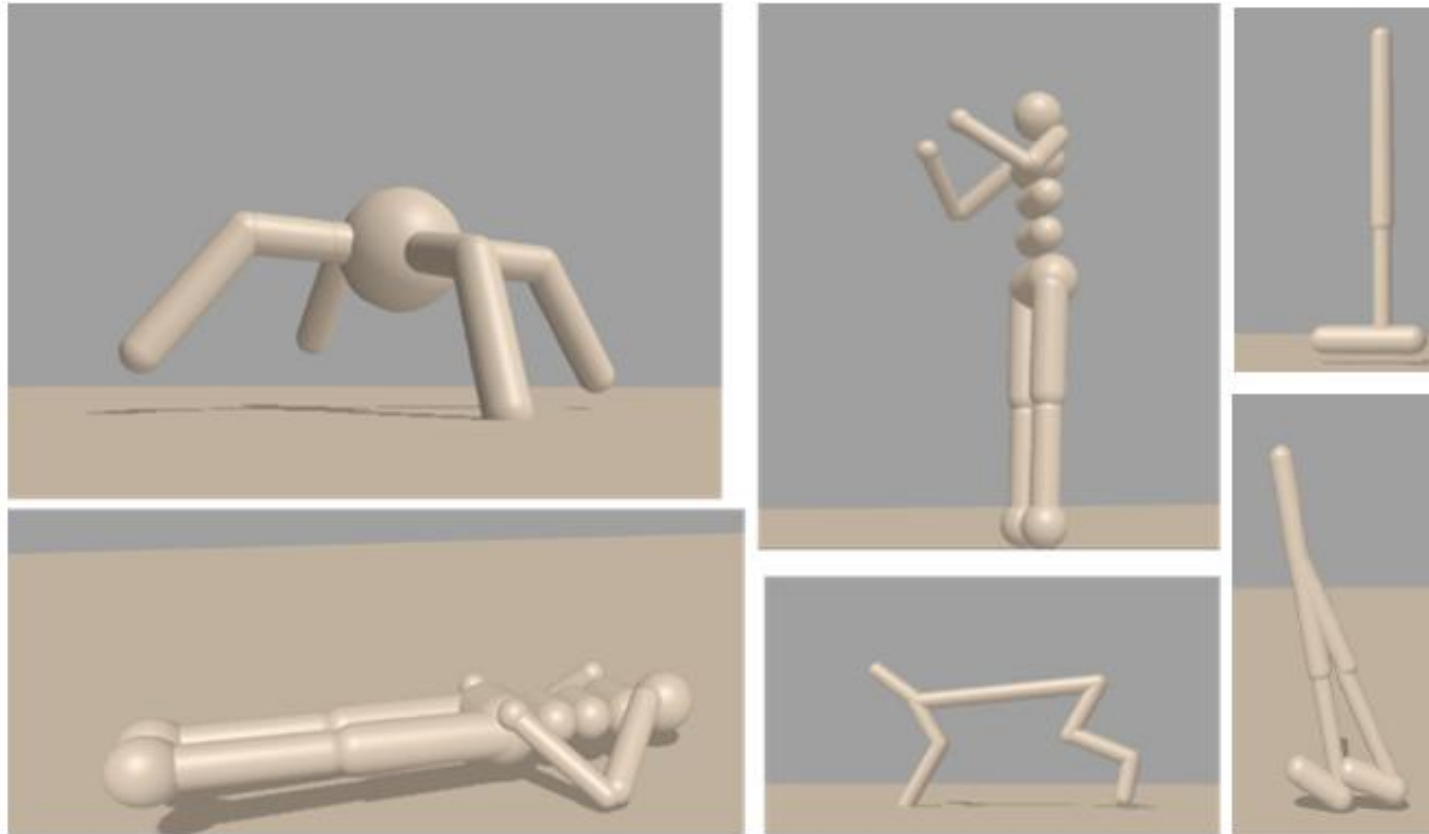
Examples of Framework Components

Inverted double pendulum environment



Examples of Framework Components

Environments: Ant, Humanoid, Hopper,
HumanoidStandup, HalfCheetah, and Walker2D



Experiments & Results Analysis

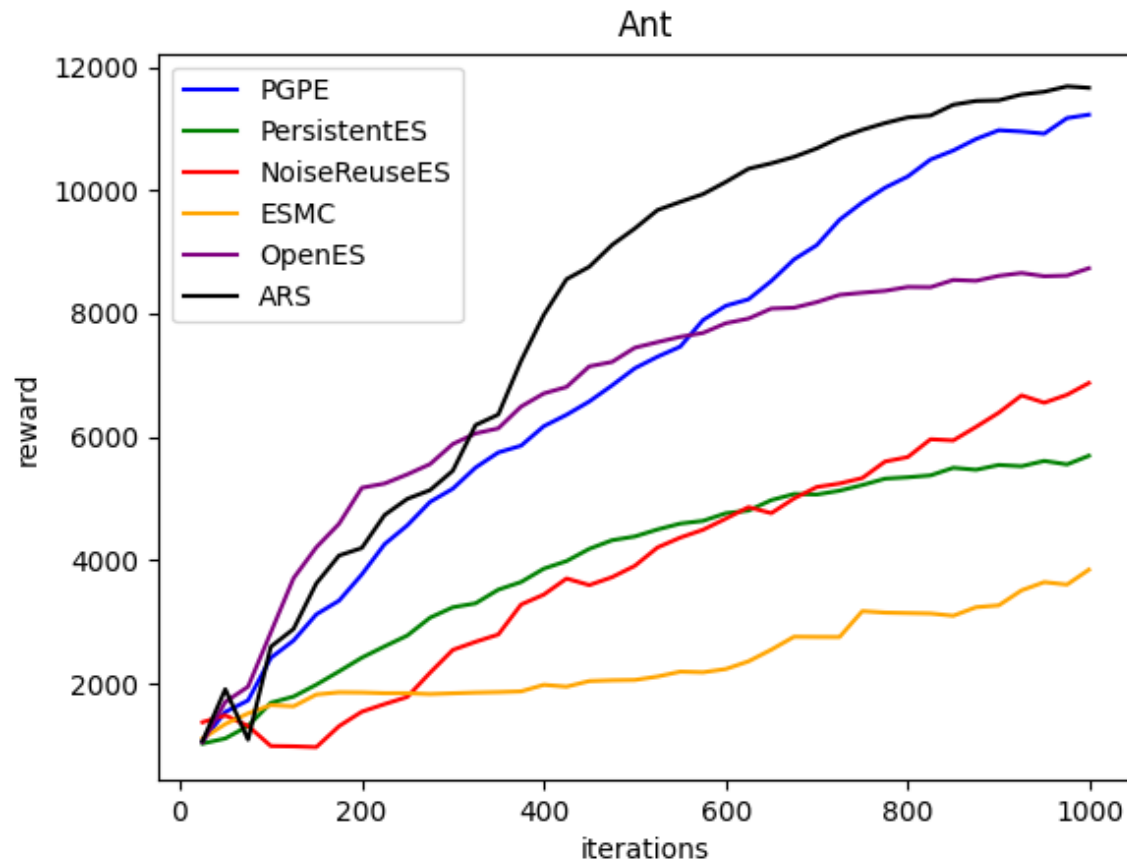
Experiments

- Conducting 4 different sets of experiments:
 - 6 different algorithms with 6 different environments
 - 6 different environments with 4 different population sizes
 - 4 different algorithms with 6 different activation functions
 - 4 different population sizes with different number of iteration on CPU vs GPU

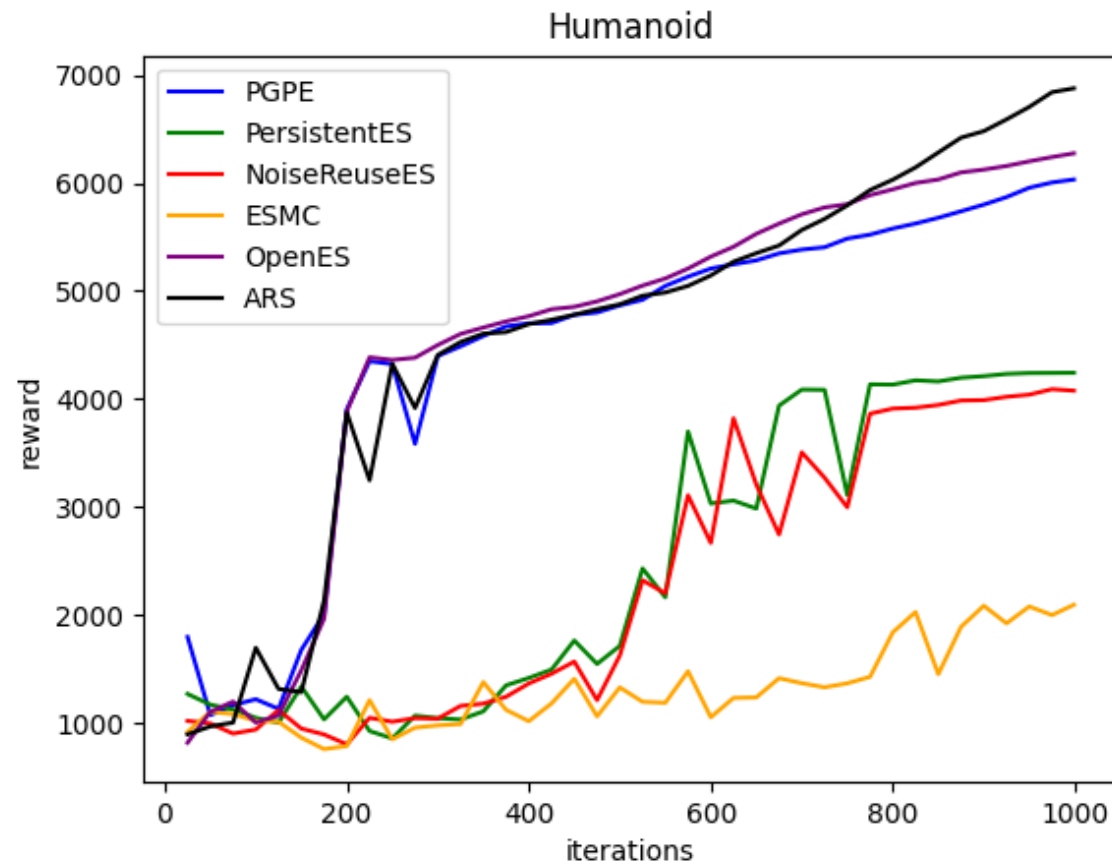
Different Environments with Different Algorithms

- 36 experiments
- 6 different evolutionary algorithms
 - PGPE, PersistentES, NoiseReuseES, ESMC, OpenES, and ARS
- 6 different environments
 - Ant, Humanoid, HumanoidStandup, Walker2D, HalfCheetah, and Hopper
- MLP neural network
 - 4 hidden layers with 64 node in each layer
 - tanh activation function
- 1024 population size
- 1000 iterations

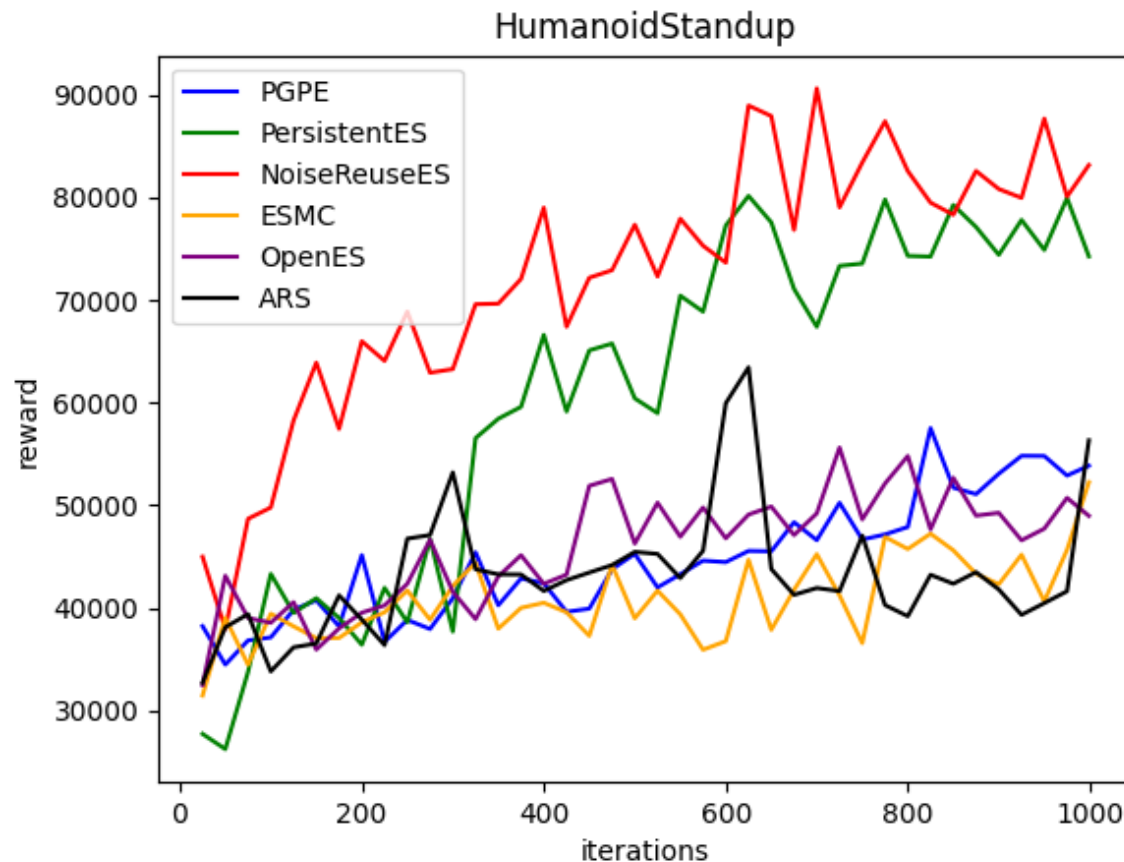
Different Environments with Different Algorithms



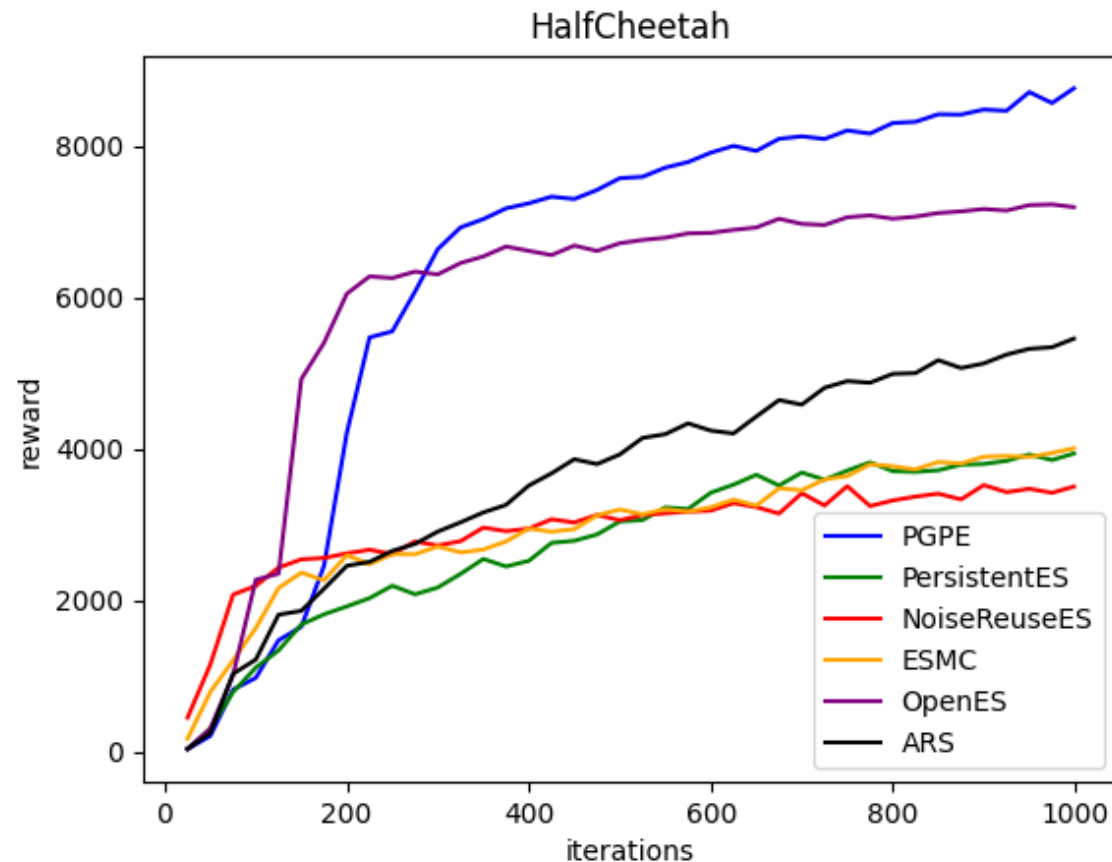
Different Environments with Different Algorithms



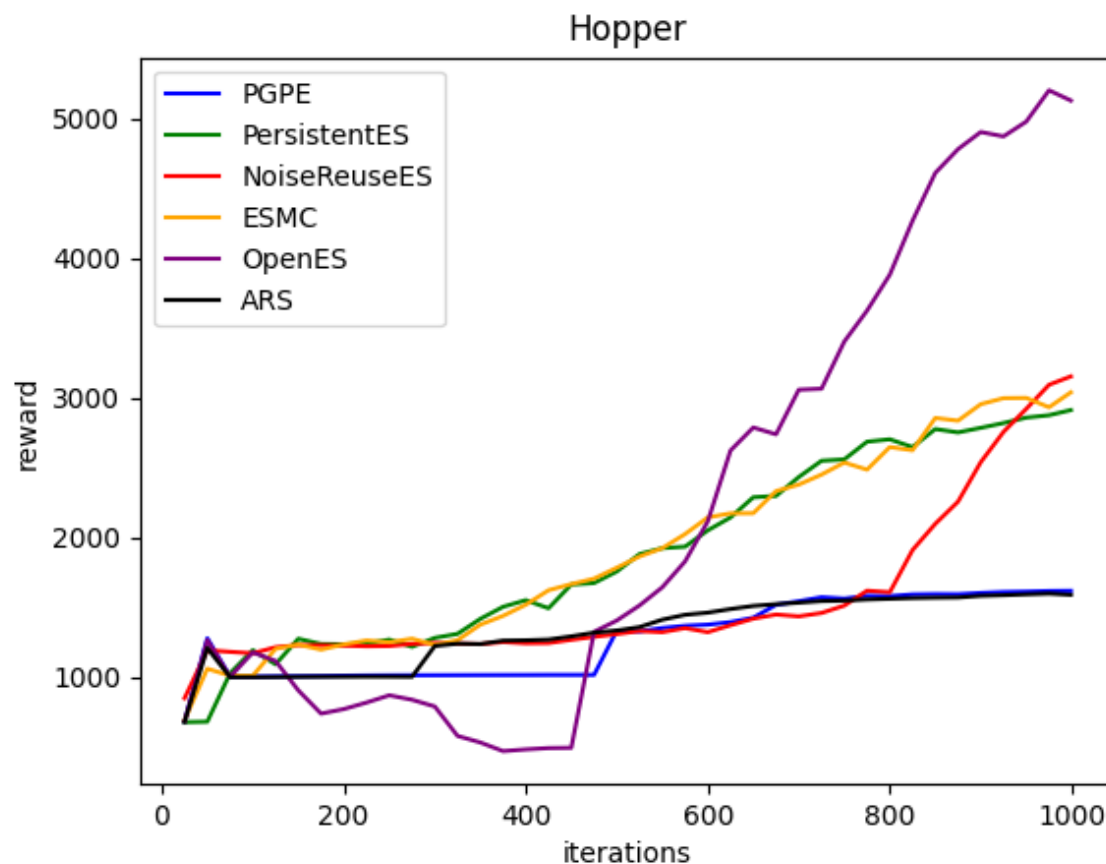
Different Environments with Different Algorithms



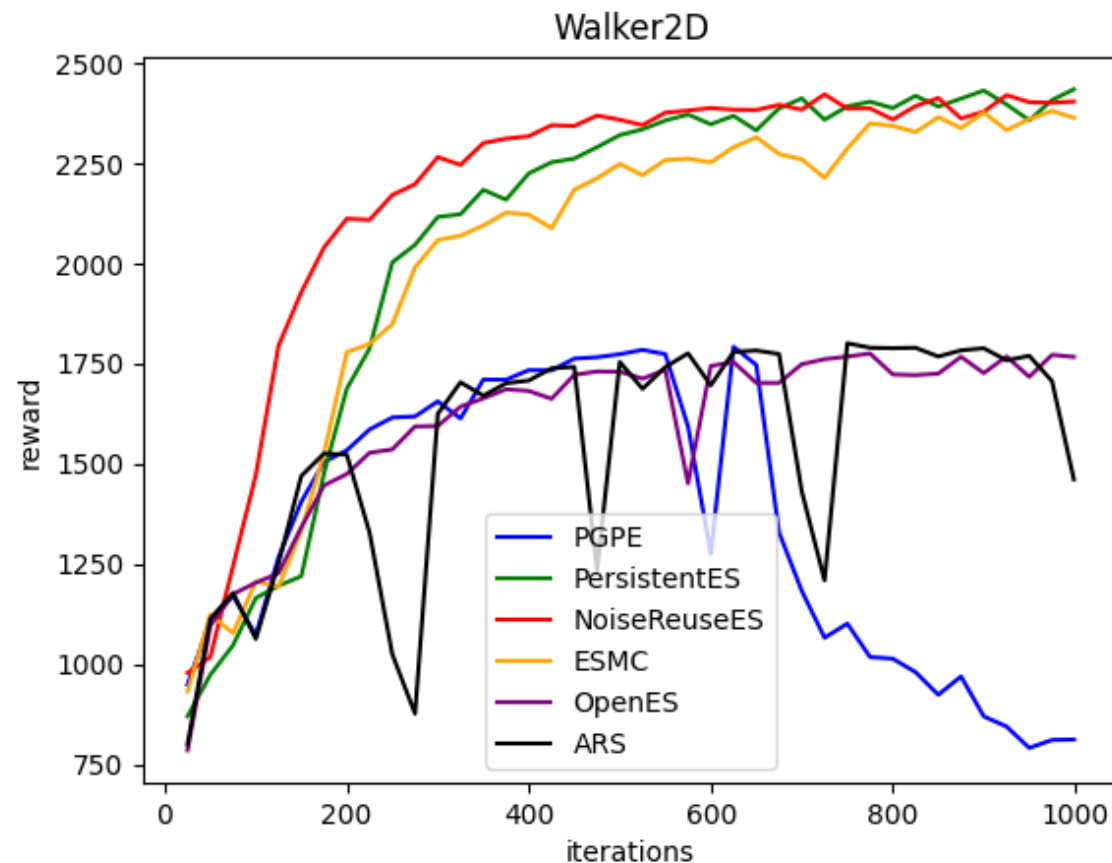
Different Environments with Different Algorithms



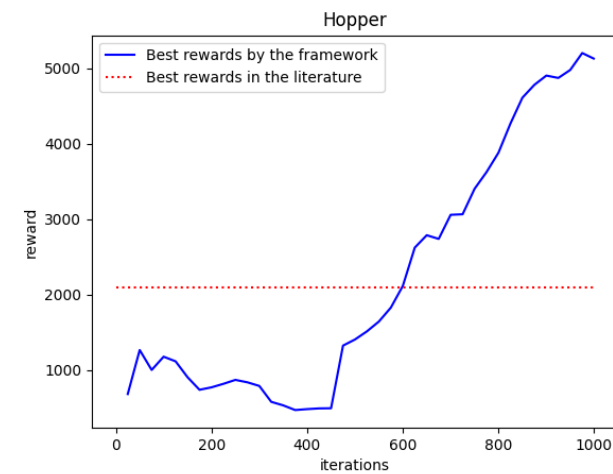
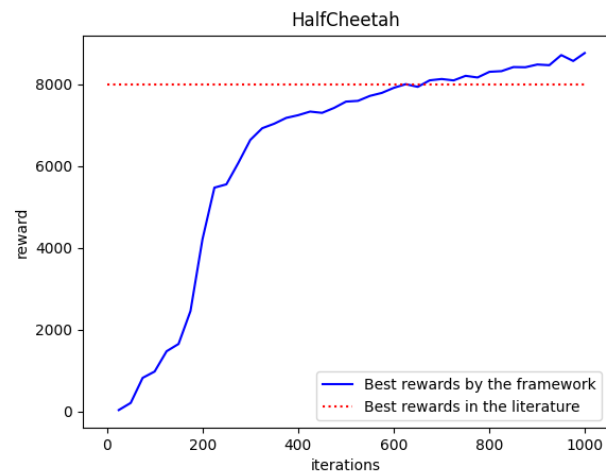
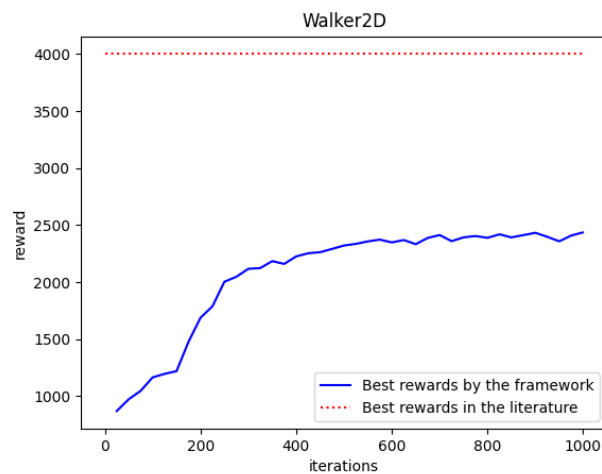
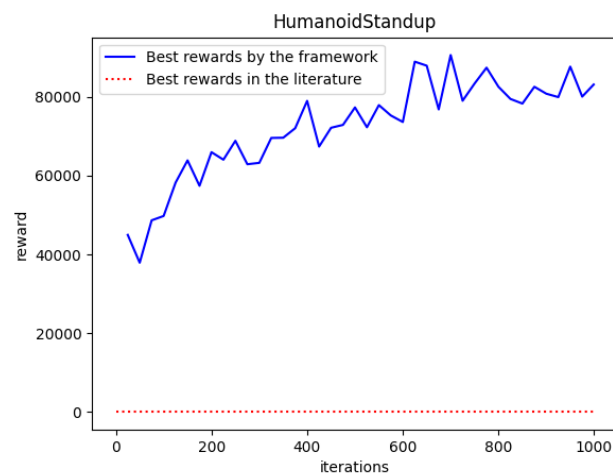
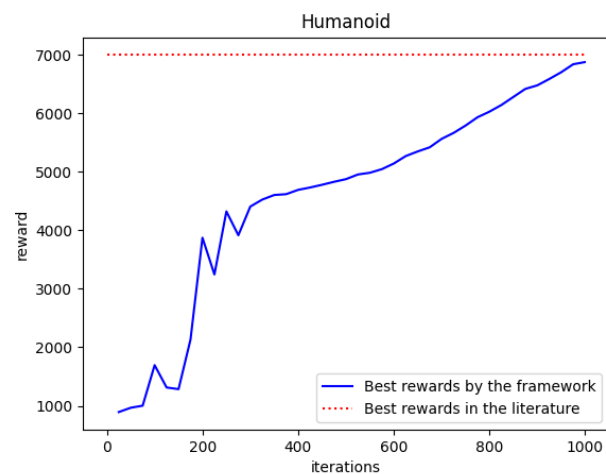
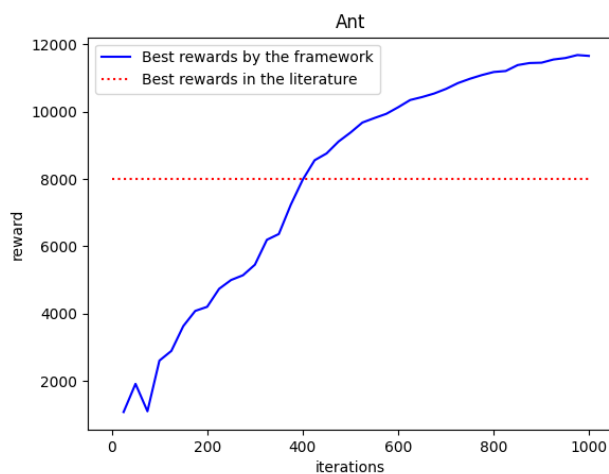
Different Environments with Different Algorithms



Different Environments with Different Algorithms



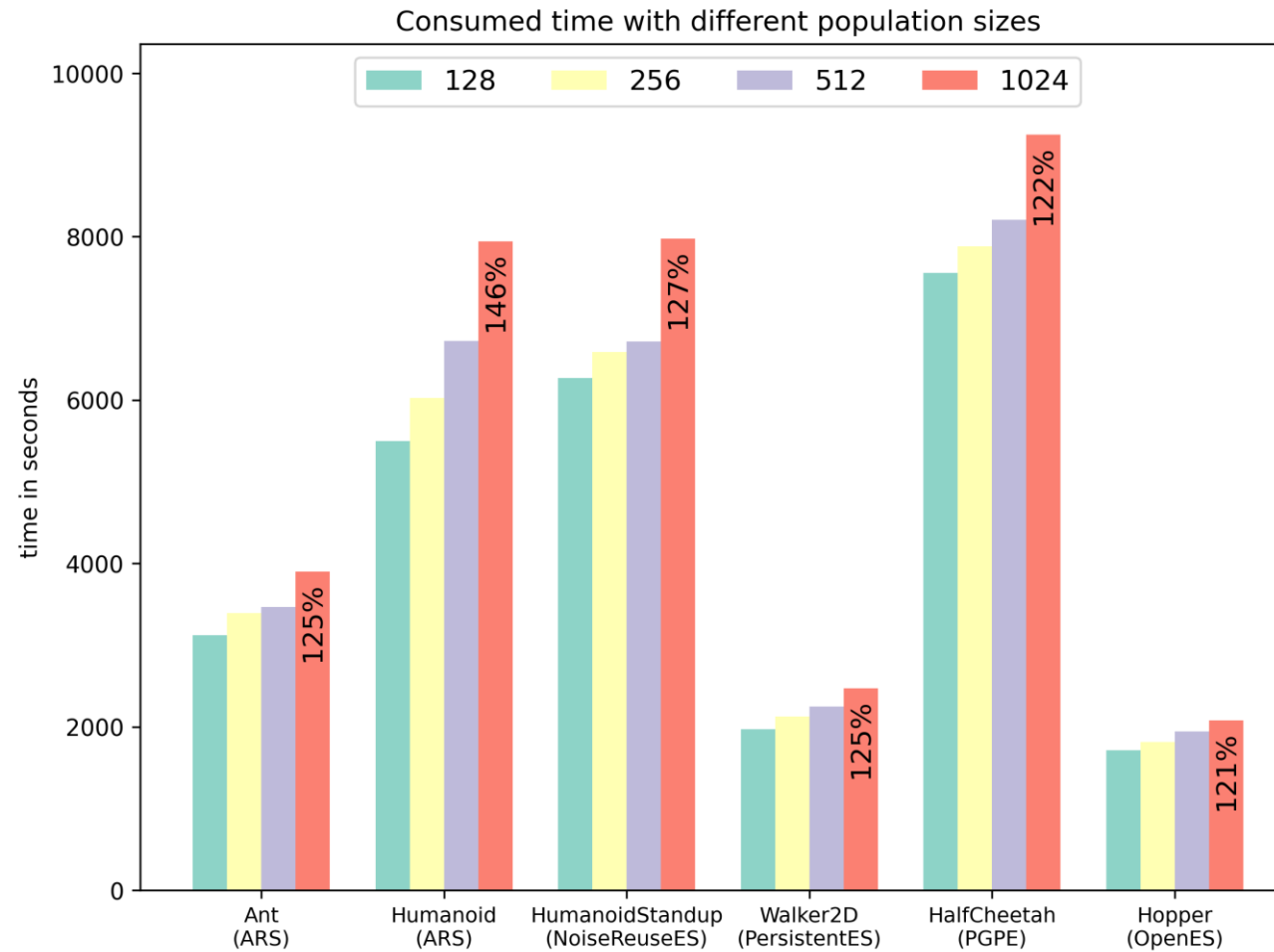
Rewards Comparison



Different Population Sizes

- 24 experiments
- 6 different environments
 - Ant, Humanoid, HumanoidStandup, Walker2D, HalfCheetah, and Hopper
- 4 different population sizes
 - 128, 256, 512, and 1024
- The algorithm with the highest reward for each environment
- MLP neural network
 - 4 hidden layers with 64 node in each layer
 - tanh activation function
- 1000 iterations

Different Population Sizes



Different Activation Functions

- 24 experiments
- Inverted Double Pendulum environment
- 4 different evolutionary algorithms
 - NoiseReuseES, PersistentES, PGPE, and ARS
- MLP neural network
 - 2 hidden layers with 256 node in each layer
 - 6 different activation functions
 - CELU, SELU, ReLU, GELU, SiLU, and tanh
- 512 population size
- 300 iterations

Different Activation Functions

○ NoiseReuseES

- generally stable under all activation functions
- most stable under SELU activation function

○ PersistentES

- unstable under all activation functions
- less unstable under GELU activation function

Different Activation Functions

○ PGPE

- stability varies under different activation functions
- stable under CELU and tanh activation function
- unstable under other activation functions

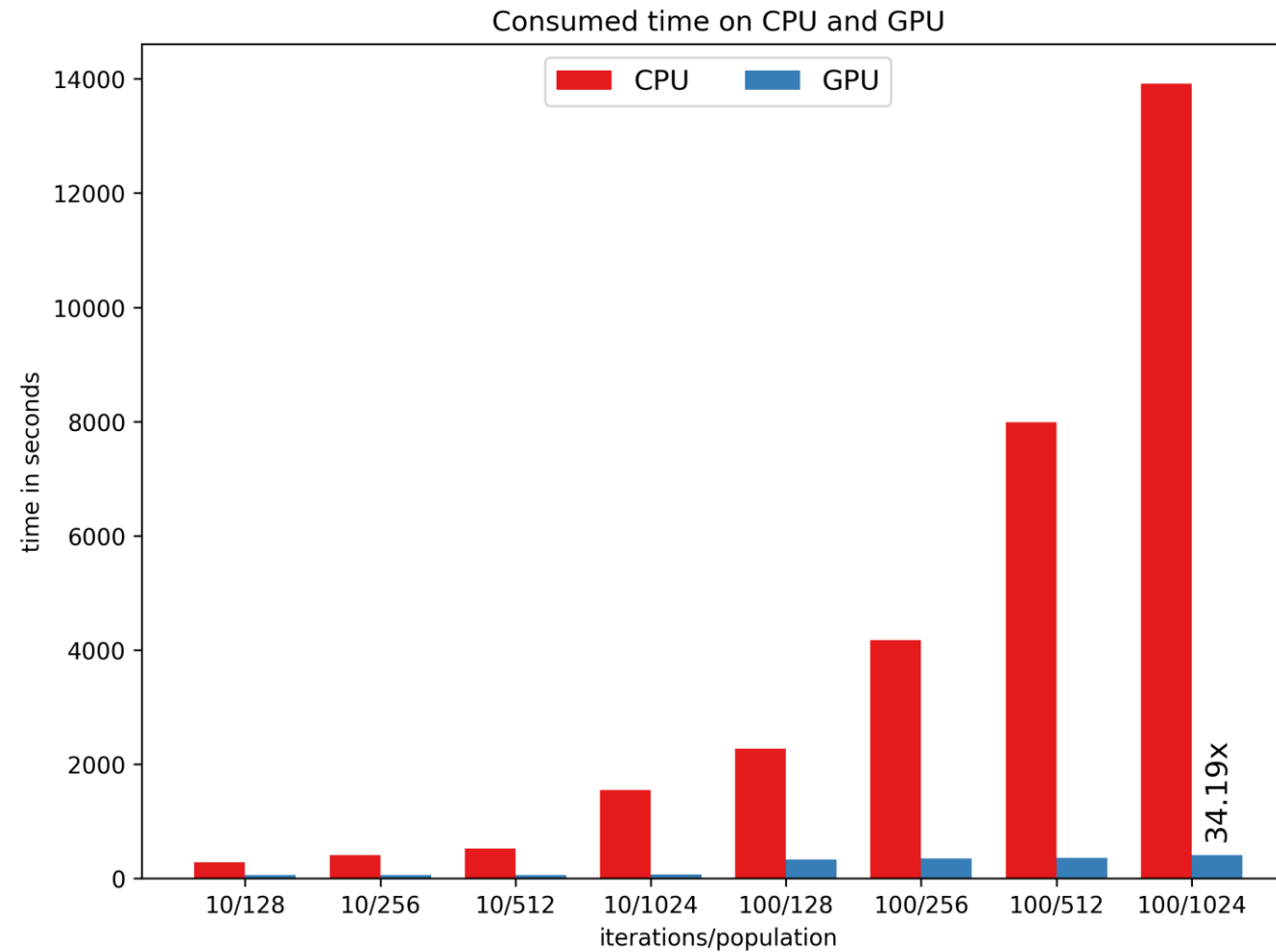
○ ARS

- unstable under most activation functions
- most stable under GELU activation function

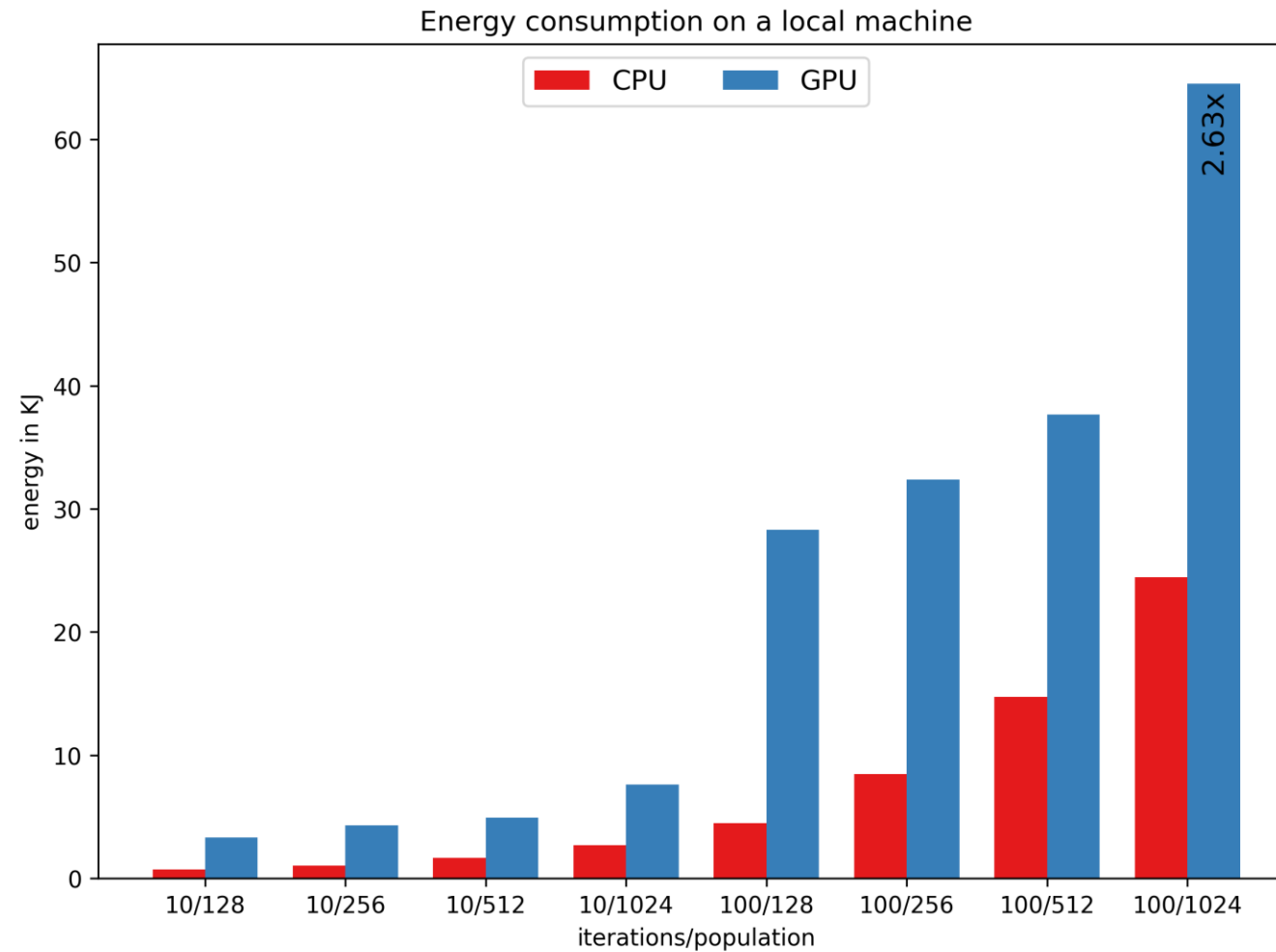
Performance on CPU vs GPU

- Ant environment
- ARS algorithm
- MLP neural network
 - 4 hidden layers with 64 node in each layer
 - tanh activation function
- 4 different population sizes
 - 128, 256, 512, and 1024
- 2 different number of iterations
 - 10, and 100

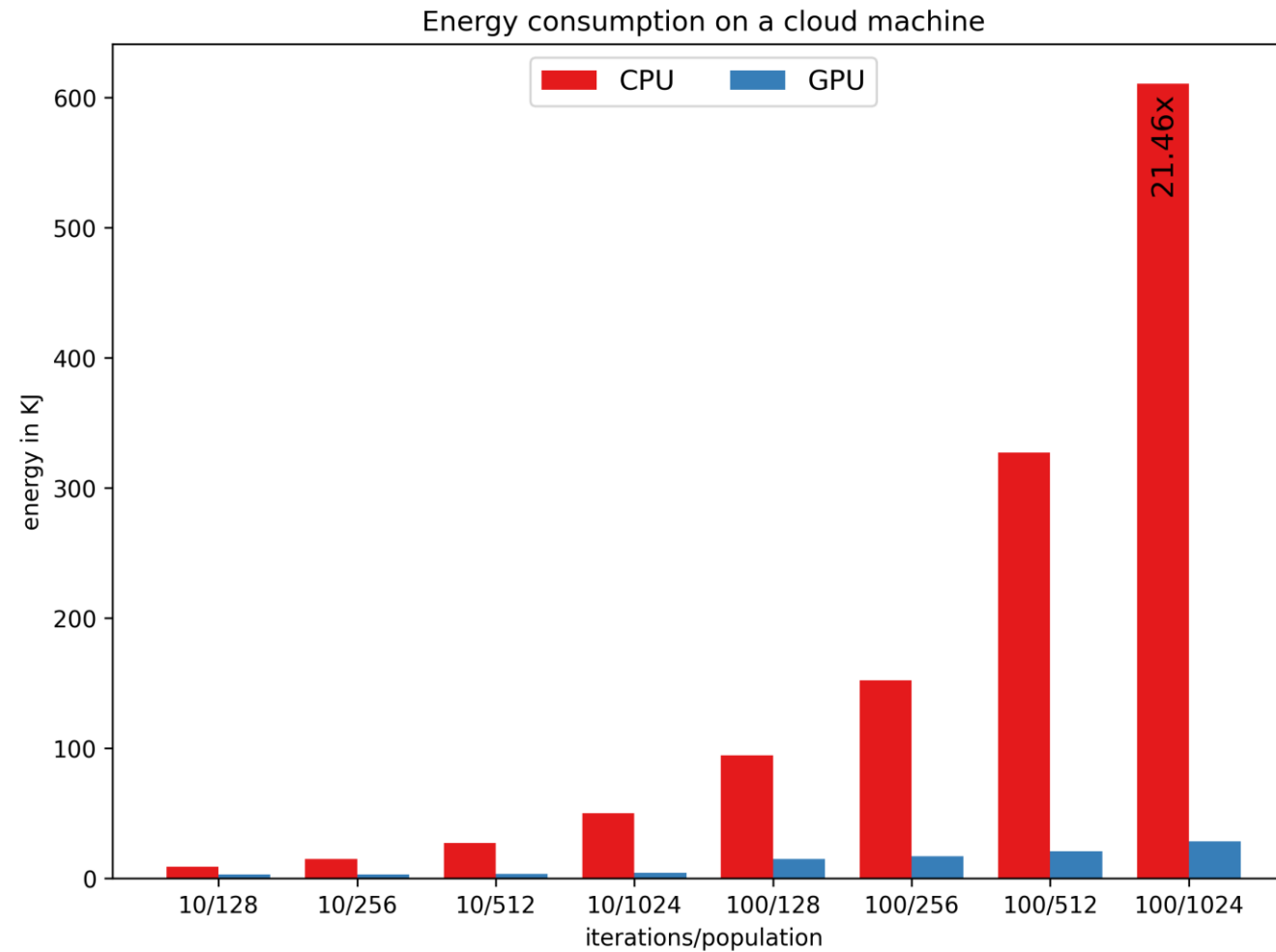
Performance on CPU vs GPU



Performance on CPU vs GPU



Performance on CPU vs GPU



Conclusion & Future Work

Conclusion

- Introducing a new framework for NeuroEvolution Deep Reinforcement Learning
 - diverse, flexible, scalable and easy-to-use
- Conducted experiments demonstrated that:
 - the proposed framework is highly scalable
 - increasing population size from 128 to 1024 (800%)
 - only increases the time consumed by 121%~146%
 - achieving a speedup factor of 34.19 times faster on GPU vs CPU

Conclusion

- Conducted experiments demonstrated that:
 - different algorithms achieved better results with different environments
 - ARS algorithm with Ant and Humanoid environments
 - NoiseReuseES algorithm with HumanoidStandup environment
 - PGPE algorithm with HalfCheetah environment
 - OpenES algorithm with Hopper environment
 - PersistentES algorithm with Walker2D environment

Conclusion

- Conducted experiments demonstrated that:
 - different algorithms achieved better results with different activation functions
 - NoiseReuseES algorithm with SELU activation function
 - PersistentES algorithm with GELU activation function
 - PGPE algorithm with CELU and tanh activation functions
 - ARS algorithm with GELU activation function

Future Work

- Experiments with different combinations
- Adding new algorithms
- Adding new environments
- Adding new neural network architectures
- Adding new training paradigm
 - multi-objective evolutionary algorithms
 - multi-agent reinforcement learning environments

Thank you

Presented by: Hesham Magdy